

W14D02 — CRUD Operations via ORM

ML Engineer Journey · Phase 4 — Database & Observability · 9 Jun 2026

1. Analogi: Buku Agenda dan Lemari Arsip

Bayangkan SQLAlchemy Session sebagai **buku agenda** dan database sebagai **lemari arsip permanen**. Kamu menulis nama di agenda dulu — itulah `session.add()`. Nama itu ada di RAM Python, belum di disk. Saat `flush()`, kamu kirim draft ke meja kasir — SQL sudah dikirim, transaksi terbuka, tapi bisa dibatalkan. Saat `commit()`, dokumen di-cap stempel, dimasukkan ke lemari arsip permanen — tidak bisa dibatalkan.

2. State Lifecycle Object SQLAlchemy

Setiap object ORM punya **state** yang berubah selama lifecycle-nya. Memahami ini kritis untuk debugging — terutama kenapa UUID bisa `None` atau kenapa perubahan tidak tersimpan.

State	Cara masuk	Apa yang terjadi	Ada di DB?
Transient	<code>obj = Model(...)</code>	Dibuat di Python, session tidak tahu	Tidak
Pending	<code>session.add(obj)</code>	Session track, belum ada SQL	Tidak
Persistent	<code>flush()</code> atau <code>commit()</code>	SQL dikirim, ID terisi, tracked oleh session	Ya (in-flight/permanen)
Detached	<code>session.close()</code> / <code>expunge()</code>	Object 'terlepas', tidak di-sync lagi	Ya (tapi stale)
Deleted	<code>session.delete(obj) + commit()</code>	Dihapus dari DB	Tidak

Jebakan paling umum: Kamu ambil object dari session A, session ditutup, lalu kamu akses relationship-nya di luar session — **DetachedInstanceError**. Object sudah Detached, SQLAlchemy tidak bisa lazy-load lagi karena tidak ada koneksi aktif.

3. flush() vs commit() — Perbedaan yang Sering Bikin Bingung

Ini adalah sumber kebingungan nomor satu untuk developer yang baru pakai ORM. Keduanya 'mengirim' sesuatu ke database — tapi maknanya sangat berbeda.

	flush()	commit()
SQL dikirim?	Ya — INSERT/UPDATE/DELETE dikirim	Ya — flush otomatis dulu, lalu commit
Transaksi ditutup?	Tidak — masih in-flight, bisa rollback	Ya — transaksi ditutup, permanen
Data terlihat koneksi lain?	Tidak (isolation level default)	Ya — semua koneksi bisa baca
UUID / server_default terisi?	Ya — Postgres sudah jalankan <code>gen_random_uuid()</code>	Ya — sama
Kapan pakai?	Perlu ID sebelum commit (buat relasi), atau setelah insert biar bisa relasi	Setelah insert selesai, atau sebelum aksi lain

Kenapa flush() sebelum append child? Saat kamu buat parent dan langsung append child ke relationship, child butuh `parent.id` sebagai foreign key. Kalau belum di-flush, `parent.id` masih `None` (`gen_random_uuid()` belum dipanggil Postgres). `flush()` memaksa Postgres generate UUID dulu tanpa menutup transaksi.

4. CREATE — Menyimpan Object Baru

Pola paling aman untuk CREATE, termasuk relasi parent-child dalam satu transaksi:

```
with Session(engine) as session:

    parent = AnalysisRequest(title="Simulasi Q2", parameters={...}, status="processing")

    # Append child via relationship – SQLAlchemy set foreign key otomatis
    for s_type in ["optimistic", "pessimistic"]:
        child = ScenarioResult(scenario_type=s_type, risk_score=0.5, output_data={})
        parent.scenarios.append(child)

    session.add(parent)    # Cukup add parent, cascade handle child
    session.flush()        # Postgres generate UUID sekarang, transaksi masih terbuka
    print(parent.id)       # UUID sudah terisi di sini

    session.commit()       # Kunci permanen ke disk
```

Kenapa tidak perlu `session.add(child)`? Karena `relationship` di model punya `cascade='all, delete-orphan'`. SQLAlchemy propagate `add()` ke semua object yang terhubung via `relationship` secara otomatis.

5. READ — Query dengan SQLAlchemy 2.x

Ada dua cara utama membaca data — pilih yang tepat sesuai kebutuhan:

```
# Method 1: get() – untuk primary key lookup, PALING EFISIEN
# SQLAlchemy cek identity map (cache) dulu sebelum ke DB
obj = session.get(AnalysisRequest, some_uuid) # return None jika tidak ada

# Method 2: select() – untuk query fleksibel
from sqlalchemy import select

stmt = (
    select(AnalysisRequest)
    .where(AnalysisRequest.status == "processing")
    .order_by(AnalysisRequest.created_at.desc())
    .limit(10)
)

results = session.scalars(stmt).all() # list of AnalysisRequest objects
```

	<code>session.get(Model, pk)</code>	<code>select(Model).where(...)</code>
Gunakan untuk	Primary key lookup	Query dengan kondisi / sort / limit
Cache	Cek identity map dulu	Selalu ke DB
Return	Object atau None	Result — perlu <code>.scalars().all()</code>
SQL setara	SELECT WHERE id = ?	SELECT WHERE ... ORDER BY ... LIMIT ...

Kenapa `.scalars().all()`? `session.execute()` return objek *Result* generik — bisa berisi tuple kolom, bukan object ORM langsung. `.scalars()` mengambil kolom pertama (yaitu object ORM-nya). `.all()` konversi jadi list Python biasa.

6. UPDATE — Unit of Work Pattern

Ini yang paling elegan dari ORM — kamu tidak perlu tulis SQL UPDATE sama sekali. Cukup ubah atribut object, SQLAlchemy yang generate SQL-nya saat commit.

```
with Session(engine) as session:
    req = session.get(AnalysisRequest, target_id)
    if req:
        req.status = "completed"    # Cukup ini – SQLAlchemy track perubahan otomatis
        # Tidak perlu session.add() lagi – object sudah Persistent
        session.commit()
        # SQL yang digenerate: UPDATE analysis_requests SET status='completed',
        #                               updated_at=now() WHERE id = ?
```

Dirty Tracking: SQLAlchemy memantau setiap atribut yang berubah dari object Persistent. Saat commit, hanya kolom yang benar-benar berubah yang di-UPDATE. Kalau kamu set `req.status = req.status` (nilai sama), tidak ada SQL yang dikirim.

7. DELETE — Cascade dan Performa

Bagian yang paling banyak jebakan tersembunyi:

```
with Session(engine) as session:
    req = session.get(AnalysisRequest, target_id)
    if req:
        session.delete(req)    # Mark sebagai 'to be deleted'
        session.commit()       # DELETE SQL dikirim ke DB + cascade ke child
```

Yang terjadi di balik layar saat delete parent:

Langkah	Apa yang dilakukan SQLAlchemy	Mengapa
1	SELECT ke semua related tables (audit_logs, scenarios, results, child)	Untuk memelihara ke identity map — cascade='all, delete-orphan' butuh info ini
2	DELETE child rows satu per satu lewat Python	SQLAlchemy hapus child dari identity map sebelum hapus parent
3	DELETE parent row	Foreign key constraint terpenuhi karena child sudah hilang
4	COMMIT	Semua perubahan dikunci permanen

Masalah performa: Kalau parent punya 100.000 child rows, langkah 1 dan 2 akan load SEMUA child ke RAM Python — bisa OOM di production. **Solusi:** tambahkan `passive_deletes=True` pada relationship DAN `ondelete='CASCADE'` pada ForeignKey. Dengan ini SQLAlchemy skip SELECT dan biarkan Postgres yang hapus child secara native — jauh lebih efisien.

Di models.py – kombinasi yang benar untuk performa:

```
scenarios = relationship(
```

```

        "ScenarioResult",
        back_populates="request",
        cascade="all, delete-orphan", # Python-level cascade
        passive_deletes=True          # Jangan SELECT child saat delete parent
    )

# Di ForeignKey:
request_id = mapped_column(
    UUID(as_uuid=True),
    ForeignKey("analysis_requests.id", ondelete="CASCADE") # DB-level cascade
)

```

8. N+1 Problem — Jebakan Lazy Loading

Ini adalah bug performa paling silent dan paling sering terjadi di aplikasi yang pakai ORM. Terlihat benar, berjalan lambat, dan tidak ada error sama sekali.

```

# BERBAHAYA – N+1 Query

requests = session.scalars(select(AnalysisRequest)).all() # 1 query

for req in requests:

    print(req.scenarios) # <-- 1 query PER request! Kalau ada 100 request = 101 query total

# AMAN – Eager Loading dengan selectinload

from sqlalchemy.orm import selectinload

stmt = select(AnalysisRequest).options(selectinload(AnalysisRequest.scenarios))

requests = session.scalars(stmt).all() # 2 query total: 1 untuk parent, 1 untuk semua child

```

Kapan N+1 terjadi? Selalu saat kamu loop query result dan di dalam loop akses relationship tanpa eager load. Di development tidak terasa karena data sedikit. Di production dengan ribuan row, ini bisa bikin response time naik 10-100x.

9. Ringkasan: ORM vs Raw SQL

Operasi	ORM	Raw SQL setara
Create	session.add(obj); commit()	INSERT INTO ... RETURNING id
Read by PK	session.get(Model, pk)	SELECT WHERE id = ?
Read query	select(Model).where(...).scalars().all()	SELECT WHERE ... ORDER BY ... LIMIT ...
Update	obj.attr = val; commit()	UPDATE SET ... WHERE id = ?
Delete	session.delete(obj); commit()	DELETE WHERE id = ?
Advantage ORM	Type-safe, dirty tracking, relationship navigasi, cascade	
Advantage SQL	–	Kontrol penuh, lebih efisien untuk bulk ops

10. Hal yang Paling Sering Dilupakan

- **session.refresh(obj) setelah commit** — jika model pakai server_default (UUID, timestamp), nilainya diset Postgres saat INSERT. Tanpa refresh, atribut itu masih None atau stale di Python.
- **DetachedInstanceError** — mengakses relationship di luar session. Solusi: akses semua data yang dibutuhkan di dalam blok 'with Session()' atau gunakan eager loading.
- **flush() di dalam delete tidak perlu** — jika pakai context manager yang auto-commit, flush manual hanya perlu jika kamu butuh validasi constraint dalam satu transaksi yang sama.
- **session.get() vs query** — get() cek identity map (cache) dulu, query tidak. Untuk primary key lookup, selalu pakai get() — lebih cepat dan tidak redundant.
- **Jangan campurkan session antar request di FastAPI** — setiap HTTP request harus punya session sendiri. Session di-share antar request = data corruption dan race condition.